Tutorial Redis

Sistemas de Big Data

Lanzar Redis

Usaremos una máquina de docker:

docker run -d --name mi_redis -p 6379:6379 -p 8001:8001 redis/redis-stack:latest

Si queremos conectarnos sólo debemos lanzar el comando:

docker exec -it mi_redis redis-cli

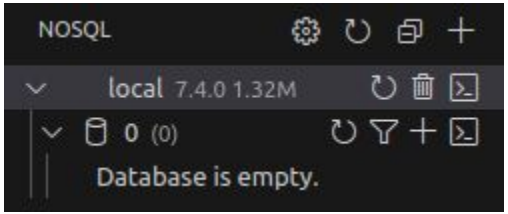


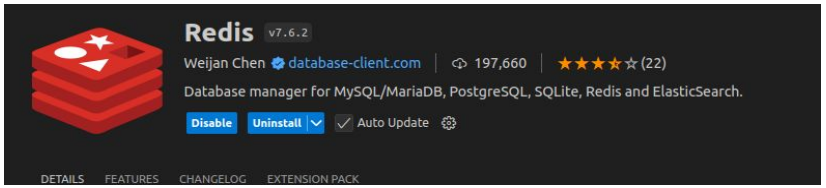
Extensión Redis Vscode

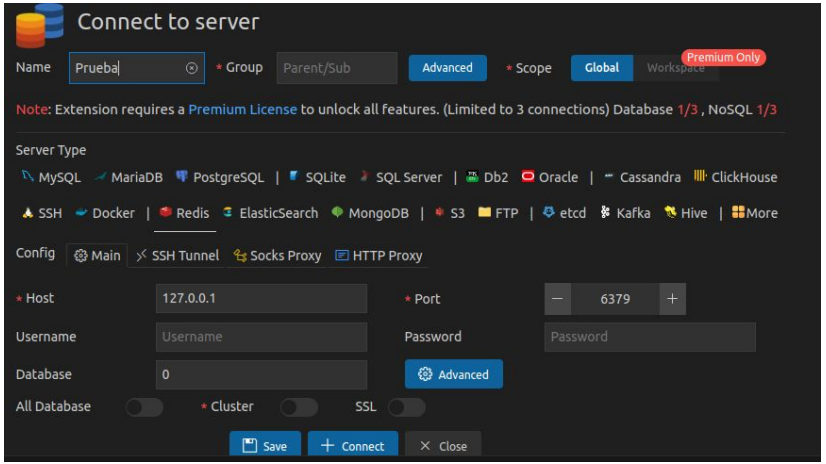
También podemos conectarnos con la extensión de redis de VsCode.

Solo hay que añadir una conexión y configurarla, con los datos correctos, y nos debe aparecer a la izquierda nuestra base de datos.









Preparar entorno Python

Antes de ponernos a programar en python, es necesario explicar como crear los entornos virtuales para evitar problemas de paquetes y de entornos:

python3 -m venv redis-env -> Creamos el entorno

source redis-env/bin/activate -> Activamos el entorno

Creamos un fichero **requirements.txt** en la misma ruta que redis-env -> Incluiremos las librerías de python que queremos. En nuestro caso tendrá la siguiente línea -> **redis~=7.0.1**

python -m pip install --upgrade pip

pip install -r requirements.txt -> instalamos los paquetes que necesitamos

Documentación: https://pypi.org/project/redis/

https://redis.io/docs/latest/develop/clients/redis-py/

Código Python para conectarnos

Importamos la librería:

```
import redis
```

Creamos la conexión a Redis:

```
conexionRedis = redis.ConnectionPool(host='localhost', port=6379, db=0)

baseDatosRedis = redis.Redis(connection_pool=conexionRedis)
```

Si queremos que la conexión decodifique directamente:

```
conexionRedis = redis.ConnectionPool(host='localhost', port=6379, db=0,decode_responses=True)
```

Código para crear registros

Para crar registros en la base de datos sólo debemos lanzar el comando **set**:

```
baseDatosRedis.set('libro_1', 'Quijote')

baseDatosRedis.set('libro_2', 'Hamlet', ex=10)
```

El segundo parametro **ex** es para indicar el tiempo de vida en segundo de esa clave en la base de datos.

Código para obtener valores

Obtener el valor de la clave "libro_1" pero en binario:

baseDatosRedis.get("libro_1")

Obtener el valor de la clave "libro_1" pero en String(Al tener la decodificación ya activada no nos haría falta)

baseDatosRedis.get("libro_1").decode("utf-8")

Obtener el valor de la clave "libro_2" que al tener tiempo de vida estará ya vacía

baseDatosRedis.get("libro_2")

Actualizar valores en Redis

Para actualizar el valor de una clave, sólo debemos lanzar de nuevo el comando **set** con un valor diferente:

```
baseDatosRedis.set("libro_1","El señor de los anillos")
```

Eliminar Claves

Para eliminar claves-valor de la base de datos:

```
baseDatosRedis.delete("libro_1")

baseDatosRedis.delete("libro_2")
```

Obtener Todas las claves

Para obtener todas las claves de una base de datos redis:

```
claves = baseDatosRedis.keys()
```

Si queremos mostrar la información de todas las claves:

```
for clave in claves:
    print('Clave:', clave , ' y Valor: ', baseDatosRedis.get(clave))
```

Buscar por las claves

Ahora utilizamos la función **scan_iter**, que recibe por parámetro un patrón en las claves para obtener sólo las que necesitamos:

```
baseDatosRedis.set("libro_1","Quijote")
baseDatosRedis.set("libro_2","Hamlet")
baseDatosRedis.set("libro_3","Otelo")
baseDatosRedis.set("comic_1","Mortadelo y Filemón")
baseDatosRedis.set("comic_2","Superman")

print("Los Libros:")
for clave in baseDatosRedis.scan_iter('libro*'):
   print(clave)

print("Los Comics:")
for clave in baseDatosRedis.scan_iter('comic*'):
   print(clave)
```

El patrón puede ir acompañado por:

- * -> Cualquier número de caracteres o ninguno
- ? -> Un Carácter
- [] -> Que contenga los caracteres dentro de los corchetes

Redis JSON

Podemos guardar archivos JSON en nuestra base de datos Redis:

```
res1 = baseDatosRedis.json().set("usuarios:1", "$", {"nombre": "Jorge", "apellido": "Baron", "edad": 37})

res2 = baseDatosRedis.json().set("usuarios:2", "$", {"nombre": "Lucía", "apellido": "Benitez", "edad": 24})
```

O podemos guardar un array de datos en json:

```
baseDatosRedis.json().set("usuarios_array", "$", [])

res3 = baseDatosRedis.json().arrappend("usuarios_array", "$", {"nombre": "Pepe", "apellido": "Sanchez", "edad": 45})

res4 = baseDatosRedis.json().arrappend("usuarios_array", "$", {"nombre": "Calisto", "apellido": "Melibea", "edad": 67})
```

Buscar en JSON

En este caso para filtrar en JSON, usaremos la segunda forma para guardarlos, la de array:

```
res1 = baseDatosRedis.json().get("usuarios_array", '$[?(@.edad > 50)]')

res2 = baseDatosRedis.json().get("usuarios_array", '$[?(@.nombre == "Pepe")]')
```

Listas

Podemos tener listas en redis también:

Añadimos a la lista:

baseDatosRedis.lpush("usuarios:hobbies", "futbol:1")

baseDatosRedis.lpush("usuarios:hobbies", "tenis:2")

baseDatosRedis.lpush("usuarios:hobbies", "rugby:2")

Obtener todos los elementos:

baseDatosRedis.lrange("usuarios:hobbies",0,-1)

Obtener el primer elemento y eliminarlo:

baseDatosRedis.rpop("usuarios:hobbies")

Índices

Nos permitirá crear la estructura de mapeo de los diccionarios y aplicar filtros sobre ellos.

Primero importamos nuevas librerías:

```
from redis.commands.json.path import Path
from redis.commands.search.field import TextField, NumericField, TagField
from redis.commands.search.index_definition import IndexDefinition, IndexType

conexionRedis = redis.ConnectionPool(host='localhost',
                                     port=6379,
                                     db=0,
                                     decode_responses=True)
baseDatosRedis = redis.Redis(connection_pool=conexionRedis)
```

Índices

Creamos el esquema:

```
squema = (
    TextField("$.nombre", as_name="nombre"),
    TagField("$.ciudad", as_name="ciudad"),
    NumericField("$.edad", as_name="edad")
)

indexCreated = baseDatosRedis.ft("indice:usuarios").create_index(
    esquema,
    definition=IndexDefinition(
        prefix=["usuarios:"], index_type=IndexType.JSON
    )
)
```

Índices

Se añaden los datos

```
usuario1 = {
   "nombre": "Jorge Baron",
   "email": "jorge.baron@example.com",
   "edad": 38,
   "ciudad": "Spain"
}

usuario2 = {
   "nombre": "Elena Gómez",
   "email": "elena.gomez@example.com",
   "edad": 29,
   "ciudad": "Mexico"
}

usuario3 = {
   "nombre":"Pepe garcia",
   "email": "pepe.garcia@example.com",
   "edad": 33
   "ciudad": "Spain"
}

user1Set = baseDatosRedis.json().set("usuarios:1", Path.root_path(), usuario1)
user2Set = baseDatosRedis.json().set("usuarios:2", Path.root_path(), usuario2)
user3Set = baseDatosRedis.json().set("usuarios:3", Path.root_path(), usuario3)
```

Índices

Importamos las librerías necesarias:

```
from redis.commands.search.query import Query

import redis.commands.search.aggregation as aggregations

import redis.commands.search.reducers as reducers
```

Podemos hacer búsquedas con filtros:

```
resultados = baseDatosRedis.ft("indice:usuarios").search(

            Query("Jorge @edad:[30 40]")
)
```

Podemos hacer búsquedas con group by:

```
groupby = aggregations.AggregateRequest("*").group_by(

            '@ciudad', reducers.count().alias('count')
)

resultado = baseDatosRedis.ft("indice:usuarios").aggregate(groupby).rows
print(resultado)
```

Gestión de índices

Para eliminar un índice ya creado:

```
baseDatosRedis.ft("indice:usuarios").dropindex(True)
```

Para obtener información de un índice:

```
baseDatosRedis.ft("indice:usuarios").info()
```

Para obtener la lista de índices:

```
indices = baseDatosRedis.execute_command('FT._LIST')
```

Documentación

Documentación para tipos de campos:

- https://redis.io/docs/latest/develop/ai/search-and-query/indexing/field-and-type-options/

Fin

